

Descrição: Relatório apresentando as etapas de modelagem, validação dos modelos, comparação de resultados e integração com MLflow.

1. Introdução

1.1 Contextualização

O presente relatório documenta a Entrega 3, do total previsto de 4, do projeto de Machine Learning I, desenvolvido pelo grupo NorthData Consulting no âmbito da disciplina de Machine Learning I da CESAR School. O foco desta parte estará em registrar todo o desenvolvimento e aplicação de modelos de Machine Learning, em um contexto acadêmico, procurando portanto não somente apresentar os resultados, mas também trazer uma didática sobre todo o processo.

Os modelos de Machine Learning deverão ser capazes de **classificar** (tarefa de classificação) empresas em três níveis de risco ESG (Alto Risco, Risco Moderado e Baixo Risco), para alimentar uma Matriz de Criticidade do painel de BI e também **prever** (tarefa de regressão) o score numérico total de cada empresa, viabilizando o Ranking, Benchmarking e KPIs do Dashboard Principal.

Os principais achados da EDA que influenciaram a modelagem:

- **Distribuição bimodal do total_score:** O histograma revelou dois picos distintos na distribuição do total_score. Um em torno de 900 e outro em torno de 1200. O que auxiliou na definição dos limiares da variável alvo risk_level.
- **Inviabilidade da variável original total_level:** A análise revelou que a coluna total_level possui apenas duas classes no nível agregado (High e Medium), tornando-a insuficiente para alimentar a Matriz de Criticidade, que exige pelo menos três grupos de risco. Isso motivou a criação da variável derivada risk_level.
- **Score ambiental como principal determinante**: A correlação de Spearman entre environment_score e total_score foi de 0.944 que é muito alta. Confirmado pelo teste de Kruskal-Wallis ($H=146.4$, $p \approx 0.000$), que demonstrou que o setor é um preditor significativo para a dimensão ambiental. Isso orientou a inclusão da feature industry e a priorização do environment_score no pipeline.
- **Redundância entre scores, grades e levels:** A EDA confirmou que grades e levels são derivados matematicamente dos scores por intervalos fixos, mas que não determinam exatamente o score. Manter os três juntos causaria peso artificial triplo nas dimensões

ESG para o KNN (sensível a distância), distorcendo os cálculos. A decisão foi manter apenas os scores numéricos como features preditivas.

1.2. Preparação do dataset

- Variável alvo (target)

Tarefa	Variável	Tipo	Origem
Classificação	risk_level	Categórica (3 classes)	Derivada do total_score com limiares nos vales naturais da KDE: Alto Risco <900, Risco Moderado 900–1150, Baixo Risco > 1150
Regressão	total_score	Numérica contínua (600–1536)	Original do dataset

- Atributos utilizados (features)

Feature	Tipo	Justificativa
environment_score	Numérica	Principal determinante do score total (Spearman rho=0.944)
social_score	Numérica	Segunda dimensão mais determinante (rho=0.824)
governance_score	Numérica	Terceira dimensão (rho=0.714)
industry	Categórica nominal	Preditor válido para E e S —Kruskal-Wallis confirmou diferença significativa entre setores (p≈0.000)

- Atributos removidos e justificativa

Atributo	Motivo da remoção
cik, ticker, name	Identificadores únicos sem relação matemática com performance ESG —inclusão causaria decoreba do modelo
total_grade, total_level	Derivados diretos do total_score —inclusão causaria Target Leakage na classificação
environment_grade, environment_level, social_grade, social_level, governance_grade, governance_level	Redundantes com score, causariam peso artificial triplo no KNN
last_processing_date	Dado administrativo de registro sem relação com performance ESG

1.3. Principais Siglas

- CV - Cross-Validation (Validação Cruzada)
- ESG – Environmental (E), Social (S), and Governance (G)
- EDA – Exploratory Data Analysis (Análise Exploratória de Dados)
- KNN - K-Nearest Neighbors
- ML – Machine Learning
- MLOps - Machine Learning Operations

2. Estratégia de solução

A seguir, levantado principais conceitos e premissas que irão guiar as decisões a serem utilizadas na prática.

2.1. Entendimento do Problema e Preparação dos Dados

2.1.1 Definição da tarefa de ML

Antes de dar início a implementação dos algoritmos de Machine Learning, é necessário definir qual é o **Targuet**, isto é, o que o modelo deve prever.

O projeto permite que seja feita tanto classificação quanto regressão a depender de qual **Targuet** escolhido. Sendo para prever dimensões de **score**, sendo um número inteiro, deve-se utilizar **Regressão**. Caso seja para prever dimensões de **level**, cuja categorias possíveis são Excellent, High, Medium ou Low, deve-se utilizar **Classificação**.

2.1.2 Tratamento dos dados

O **pipeline** de dados deste projeto utiliza a arquitetura medalhão. O ponto de partida se dará com dados tratados na **camada Silver** e deverão passar por transformações para adequá-los a linguagem matemática que exigida pelos algoritmos através de **Pré-processamento e Engenharia de Features (Feature Engineering)** sendo alocados na **camada Ouro**.

Um cuidado adicional que também deve ser tomado é quanto a prevenção de Vazamento de Dados (**Data Leakage**), quando o modelo tem acesso a informações dos dados de teste durante a fase de treinamento, de forma que consiga aprender através de respostas que não deveria ter conhecer. Desta forma, deve-se ter o cuidado de isolar (dividir) os dados de teste antes de aplicar qualquer padronizações (**StandardScale**) ou conversões categóricas (**OneHotEncode**) no setor (**industry**).

Outro ponto a se ter atenção é que conforme o trabalho anterior realizado na parte de Análise Exploratória de Dados (EDA) e Visualizações Analíticas foi identificado que:

- Há um desbalanceamento severo na dimensão **level**: Medium ~70%, High ~29%, Excellent ~1,14%, Low ~0,07%
- O **score** ambiental é o principal determinante do **score** total (correlação 0,944)
- Setor (**industry**) é preditor válido para E e S, mas não para G

- 11 outliers identificados, sendo 7 em Social e 4 em Governança

2.2. Validação

2.2.1 Hold-out

Metodologia simples de treino e teste, onde é feita uma única divisão de grupos para treino e outra para teste, sendo treino >> teste e os dados que estarão dentro de cada grupo são escolhidos de forma aleatória. As frações de 70/30 ou 80/20 em geral são ideais para conjuntos de dados de tamanho tradicional ou acadêmico. No entanto, a literatura destaca que a proporção correta depende do tamanho absoluto da base de dados, que quanto maior, menor pode ser a porção de treino. Portanto, para o dataset a ser utilizando neste projeto, uma vez que possui 722 linhas, a adoção da divisão será de 70% para treino e 30% para teste.

- **Ponto positivo:** fácil e rápida de executar.
- **Ponto negativo:** Como os dados que pertencerão a cada grupo de treino e teste serão escolhidos de forma aleatória, o resultado é influenciado pelo fator “sorte”. Será utilizado o parâmetro `stratify=y` do `scikit-learn` para garantir que as proporções das classes minoritárias sejam mantidas nas duas fatias.

2.2.2. Validação cruzada (K-Fold)

Será feita a divisão dos dados de treino em K partes (normalmente 3, 5 ou 10). O grupo de treino será composto por K-1 parte e o de teste na parte 1 restante. Girando o processo percorrendo todas as partes de forma que todas poderão ser de treino ou de teste em algum momento.

- **Ponto positivo:** Elimina a sorte e fornece o desvio padrão da sua métrica, provando a estabilidade do modelo.
- **Ponto negativo:** Requer maior processamento computacional e mais tempo de resolução.

2.2.3. Leave-One-Out (LOO):

Feito de forma similar ao K-Fold, sendo que apenas um único registro é deixado de fora para teste a cada rodada.

- **Ponto positivo:** Em situações em que a base de dados é muito pequena, separar algo em torno de 30% a 10% do total (usando Hold-out ou K-Fold) traria uma perda sensível de informação, de forma que este método se torne adequado por exemplo para treinar um modelo para um setor ultra-específico, com pouquíssimas empresas (menos de 50 dados por exemplo).
- **Ponto negativo:** Inadequado para base de dados grande, tanto devido ao peso computacional exigido, quanto e principalmente, por este modelo ser muito sensível a Outliers (que iriam gerar saltos no resultados) e retirar apenas um registro, tende a fazer com que os conjuntos de treinamento sejam muito parecidos entre si, dificultando uma generalização.

2.2.4. Otimizações

O processo de modelagem dos algoritmos de ML envolve diversos parâmetros possíveis. Na busca do melhor resultado de aprendizado e predição do modelo, deve-se encontrar quais seriam estes “melhores parâmetros”. Quando estes parâmetros não podem ser desenvolvidos ou deduzidos pelo próprio modelo, ou seja, quando na verdade o modelo depende que eles venham de uma fonte externa para realizar seu aprendizado, temos os hiperparâmetros. Não há uma regra fechada e determinística sobre quais hiperparâmetros se usar para um determinado problema a ser resolvido, entretanto há boas práticas e um entendimento de como eles afetam o comportamento do modelo, o que ajuda a reduzir o leque de opções. Entretanto, ainda assim, o acerto dos hiperparâmetros ainda vai envolver um processo iterativo de tentativas e erros.

A técnica de ***Grid Search (Busca em Grade)*** auxilia neste processo de busca dos melhores hiperparâmetros. Trata-se de uma matriz com todos os hiperparâmetros que se cogita experimentar, e o algoritmo irá fazer o teste de todas as combinações possíveis entre eles. Para cada combinação, o modelo é treinado e o seu desempenho é avaliado usando Validação Cruzada (Cross Validation).

- **Ponto positivo:** Tende a encontrar o melhor resultado possível em uma configuração ótima dentro da grade definida.
- **Ponto negativo:** Alto peso computacional exigido, que cresce exponencialmente com o aumento da quantidade de hiperparâmetros e pode gerar ineficiência de tempo tratando combinações irrelevantes.

Torna-se, portanto, a opção mais adequada quando para um reduzido espaço de busca (poucos hiperparâmetros com poucas variações) e para um dataset que não seja muito grande.

Por outro lado, técnica de ***Random Search (Busca Aleatória)*** faz uma inferência, através da escolha de amostras aleatórias dentro da grade de parâmetros que foi definida. Estabelece-se um limite de tentativas a serem feitas.

- **Ponto positivo:** Por atuar em amostras, se tem economia de tempo e recursos computacionais e ainda assim, sendo capaz de obter resultados perto do que seria o ponto ótimo. Com esta otimização, consegue-se explorar regiões amplas de hiperparâmetros economizando recursos.
- **Ponto negativo:** Por haver um certo nível de aleatoriedade, haverá sempre uma imprecisão entre os resultados de sua inferência com o resultado ótimo ao se avaliar toda a população.

Torna-se, portanto, a opção mais adequada quando se tem um espaço de busca grande, o que levaria muita complexidade ou consumo de recursos em comparação a Grid Search.

2.3. Treinamento e avaliação dos modelos

2.3.1 KNN (KNearest Neighbors)

O KNN faz a avaliação através do cálculo de distâncias no plano cartesiano para encontrar os valores mais próximos daquele que se deseja classificar ou prever (regressão). É sensível a

diferença de escalas entre as variáveis, logo, requer que seja feita normalização (*StandardScale*) nos dados numéricos antes de executar o algoritmo de KNN.

2.3.2. Árvore de Decisão (Decision Tree)

Faz a separação dos dados através de decisões binárias, seja através de perguntas (valores positivos ou falsos), seja através de critérios de comparação ou ordenamento (maior, menor). As separações são feitas de forma progressiva até se conseguir obter o grupo mais homogêneo possível. Por ser melhor interpretável, torna-se ideal para regras de negócio e apresentação em painel de BI. Porém, sofre tende a sofrer de *overfitting* (quando o algoritmo decora os dados em vez de aprender), que pode ser melhor controlado limitando o hiperparâmetro *max_depth* (profundidade da árvore).

2.3.3. Random Forest

Derivação da árvore de decisão, onde esta técnica irá gerar diversas árvores independentes e diferentes entre si. O resultado final será definido através de critérios de votação (para classificação) ou média (regressão) das árvores, absorvendo possíveis ruídos e reduzindo o problema de *overfitting*. Deve-se utilizar hiperparâmetros como *n_estimators* (número de árvores).

2.4. Rastreamento com MLflow

O MLflow é uma plataforma *open-source* para gerenciamento de ciclo de vida de projetos de *Machine Learning*(*ML Ops*).

Projetos de *Machine Learning* é comum necessitarem de inúmeras iterações de ajustes de parâmetros e tentativas e erros até se descobrir qual resultado é o de melhor desempenho.

O MLflow portanto, possui os recursos para rastrear, organizar e registrar os experimentos de forma automática e padronizada, garantindo velocidade e praticidade para elaboração de relatórios e memoriais. Possui ainda recursos de captura automática de quais algoritmos foram processados, quais os hiperparâmetros escolhidos e quais foram as métricas de erro obtidas (precisão, recall, F1-score etc).

Uma vez que o propósito deste projeto é de testar, realizar sucessivos experimentos, comparar diversos modelos, hiperparâmetros e resultados e os exibir graficamente, o seu uso, portanto, será de grande valor e deve ser feito desde o início deste projeto. Cada modelo treinado será um "run" dentro de um experimento nomeado. O MLflow salvará tudo na pasta *mlruns/* do projeto.

3. Desenvolvimento prático

3.1. Sequência de execução

Como temos uma dependência entre algumas ações, é necessário que haja um cadenciamento entre elas e portanto, o desenvolvimento prático irá seguir a seguinte sequência:

- i. Criar o notebook: 02_modeling.ipynb
- ii. Definir as Tarefas de Machine Learning e Variável Alvo (Target)
- iii. Carregar o dataset Silver e fazer o split (Hold-Out) ← primeiro corte
- iv. Configurar o MLflow ← antes de qualquer treino
- v. Construir o Pipeline Gold (dentro do notebook)
- vi. Treinar e comparar modelos ← com MLflow já rastreando
- vii. Analisar e apresentar resultados

3.2. Tarefas de Machine Learning e Variável Alvo (Target)–Primeiras considerações

- **Para Classificação:** A variável alvo será total_level, e categóricas Low/ Medium/ High/ Excellent. Sendo as métricas mais adequadas F1-Score Macro e Recall Macro. O resultado irá servir a Matriz de Criticidade e Mapa de Riscos.
- **Para Regressão:** A variável alvo será total_score, utilizando escala numérica (600 a 1536). Sendo as métricas mais adequadas RMSE, MAE e R². O resultado irá servir ao Dashboard Principal (Ranking, Benchmarking, KPIs).

3.3. Split Hold-Out

- **Percentuais treino/teste:** Será de 70/30, conforme pressimas da sessão 2.
- **Desbalanceamento:** A EDA revelou que ~70% dos dados são classe "Medium". Por isso será usado stratify=y no split da classificação com o objetivo de garantir que a proporção de cada classe seja mantida tanto no treino quanto no teste.

3.3.1. Redefinição da Variável Alvo–Risk Level

A análise da coluna total_level revelou que, no nível agregado, apenas duas classes existem no dataset: High e Medium. As classes Low e Excellent aparecem somente nas dimensões isoladas (E, S ou G). Uma classificação binária seria insuficiente para a Matriz de Criticidade, que exige pelo menos três grupos de risco (High, Medium e Low).

Conforme verificado na EDA, notamos que há um padrão que segmenta o `score` para as dimensões isoladas, em intervalos fixos de 200 pontos entre classes de `level` e de 100 pontos para classes de `grade`. Podemos então intuir que a tabela geral de score, para cada dimensão isolada, teria a seguinte estrutura:

grade	score range	level
AAA	700–800	excellent
AA	600–699	excellent
A	500–599	high
BBB	400–499	high
BB	300–399	medium
B	200–299	medium
CCC	100–199	low
CC	0–99	low

A questão é que não sabemos como a combinação de `grade` + `level` leva a pontuação (`score`). Verificamos que empresas com mesmo `grade` e mesmo `level`, podem ter `score` diferentes. Logo, considerar `grade` e/ou `level` parece não trazer ganho de informação, havendo a possibilidade de gerar redundâncias e ruído. **Consultar normas externas de como este cálculo é feito, também fugiria do objetivo deste trabalho, pois neste caso estaríamos saindo de uma análise de machine learning, que em essência busca encontrar padrões dentro de um cenário indefinido e sem "manuais ou normas de funcionamento".**

O que podemos concluir neste momento é que, sendo o `total\_score` = `environment\_score` + `social\_score` + `governance\_score`, a pontuação máxima que poderia ser atingida seria 2400. Se somássemos os scores de cada dimensão para o mesmo level, o intervalo que deveríamos ter para o `total\_score` teoricamente deveria ser:

Intervalo teórico	
level	total_score
excellent	1800-2400
high	1200-1799
medium	600-1199
low	0-599

Sendo que identificamos na prática que o intervalo para High está pelo menos entre 901-1536 Medium está pelo menos entre 600-899 do `total\_score`, o que foge do intervalo teórico. Logo, a classificação de `level` a partir do `total\_score` deve seguir uma regra diferente e a partir das informações que temos do dataset, quais os limites exatamente em que ficariam Medium, High e possivelmente Low e Excellent não está totalmente definido (4 níveis). Como os gap entre as classes são diferentes e sendo High > Medium possivelmente há alguma curva exponencial que se ajusta as classes, supondo então que os gap seriam low < medium < high < excellent. No anexo da sessão 8.1, definimos qual seria a possível divisão entre as 4 classes para uma razão fixa. Sendo que precisamos ter apenas 3 níveis categóricos para risco (alto, médio e baixo), logo, foi feito o inverso, mantendo a mesma razão, definimos então quais seriam os intervalos para cada classe.

Optou-se portanto como solução primeira, criar uma nova variável alvo chamada risk_level, derivada do total_score por meio de cortes com espaçamento exponencial. Entretanto, é importante se ter em vista que tais limiares exponenciais podem gerar classes desbalanceadas. Isso será tratado com modelos de Random Forest e avaliação por F1-Score Macro.

Categoria (risk_level)	Critério
Alto	total_score < 606
Moderado	606 <= total_score <= 1388
Baixo	total_score > 1389

Avaliando o comportamento desta divisão proposta, no dataset original, temos o seguinte comportamento:

```
Distribuicao da variavel alvo (risk_level):
```

```
risk_level
Risco Moderado    697
Baixo Risco       13
Alto Risco        12
Name: count, dtype: int64
```

```
Proporcao (%):
```

```
risk_level
Risco Moderado    96.54
Baixo Risco       1.80
Alto Risco        1.66
Name: proportion, dtype: float64
```

```
Distribuicao do total_score no dataset:
```

```
count    722.000000
mean     975.750693
std      218.751796
min       600.000000
25%      763.000000
50%     1046.000000
75%     1144.000000
max      1536.000000
Name: total_score, dtype: float64
```

```
Empresas com total_score < 606: 12
Empresas com total_score > 1388: 13
Empresas entre 606 e 1388:      697
```

```
Menor score no dataset: 600
Maior score no dataset: 1536
```

Esta decisão demonstrou um desbalanceamento extremo, com 96,54% da população no nível de risco moderado. Do ponto de vista matemático, optar por esta segmentação, teríamos maior coerência com o dataset apresentado, porém, traria pouca variedade de cenários e assim pouco ganho didático.

Desta forma, iremos tratar a solução deste projeto de forma mais abrangente, ou seja, buscar o equilíbrio entre ganho didático e comportamento do mundo real fazendo sucessivas iterações até encontrar um intervalo plausível, que traga:

- Ganhos de informações para fins didáticos
- Possibilitar uma população suficiente para que o modelo aprenda
- Ser coerente com a realidade

Assim, as premissas e restrições para testarmos os intervalos serão:

- Um intervalo de `total\_score` entre 600 - 1600 (ou aproximadamente)

- Idealmente buscar um comportamento da distribuição da população parecido com curva normal, ou seja, no centro está a maioria (medium), se possível.
- Haver pelo menos 20% de uma população dentro de uma classificação para favorecer o aprendizado

Após sucessivos processos iterativos, será usada a seguinte distribuição:

Categoria (risk_level)	Critério	Significado
Alto	total_score < 900	Performance ESG abaixo do padrão mínimo
Moderado	900 <= total_score <= 1150	Performance em desenvolvimento
Baixo	total_score > 1150	Boa maturidade ESG

Feito este ajuste, obtemos o seguinte resultado da distribuição com limiares absolutos

Classe	Empresas	Proporção
Baixo Risco	285	39,47%
Risco Moderado	271	37,53%
Alto Risco	166	22,99%

O desbalanceamento moderado é aceitável e reflete a realidade —poucas empresas atingem excelência ESG consolidada nas três dimensões simultaneamente. O F-Score Macro garantirá que o modelo seja penalizado caso negligencie a classe minoritária.

3.4. Decisão sobre features e Proteção do KNN

3.4.1. O problema da redundância dimensional

A EDA confirmou que grades e levels são derivados matematicamente dos scores por intervalos fixos. Manter os três juntos por dimensão não agrega informação nova, mas causa um problema grave para o KNN.

O KNN calcula distância Euclidiana somando a diferença de TODAS as colunas simultaneamente. Se a dimensão Ambiental estiver representada por 3 colunas (score, grade, level) e a Indústria por apenas 1, o peso do Ambiental no cálculo de distância será artificialmente 3x maior —distorcendo completamente a classificação.

Árvores de Decisão e Random Forest são imunes a esse problema pois selecionam a melhor variável em cada nó. Mas para proteger o KNN e manter consistência entre os modelos, removemos grades e levels mantendo apenas os scores contínuos.

3.4.2. Features finais

Feature	Tipo	Justificativa
industry	Categórica nominal	Preditor válido para E e S (EDA, Kruskal-Wallis)
environment_score	Numérica	Principal determinante do score total (rho=0.944)
social_score	Numérica	Segunda dimensão mais determinante (rho=0.824)
governance_score	Numérica	Terceira dimensão (rho=0.714)

Como risk_level foi derivado do total_score, que é a soma exata das três features numéricas, é provável que os modelos irão descobrir essa soma rapidamente e atingir acurácia muito alta (próxima de 100%). Esse comportamento é esperado e será documentado na seção de Discussão dos Resultados.

```
Features finais:
['industry', 'environment_score', 'social_score', 'governance_score']
```

```
Tamanho treino : 505 registros (69.9%)
Tamanho teste  : 217 registros (30.1%)
```

```
Distribuicao classes no treino (%):
```

```
risk_level
Risco Moderado    39.41
Alto Risco        37.62
Baixo Risco       22.97
Name: proportion, dtype: float64
```

```
Distribuicao classes no teste (%):
```

```
risk_level
Risco Moderado    39.63
Alto Risco        37.33
Baixo Risco       23.04
Name: proportion, dtype: float64
```

3.5. Validação Cruzada (K-fold)

Através de revisão literária, temos:

K	Características
3	Cada fold de validação tem poucos dados, risco de ter viés e uma estimativa instável
5	Equilíbrio ideal, padrão da indústria
10	Tende a ser mais preciso (mas nem sempre, devido a redução do grupo de teste). É proporcionalmente mais lento e custoso computacionalmente.

O dataset tendo 722 registros, logo a escolha de K=5 mostra-se adequado para este projeto.

3.6. ~~Leave-One-Out~~

Não utilizado. Dataset possui 722 registros, para 3 modelos, seriam 2166 treinamentos e ainda multiplicados pelas combinações do GridSearch. Teríamos um custo computacional muito alto para pouco benefício diante dos outros modelos disponíveis que trariam melhor eficiência. Conforme apontado anteriormente na sessão 2.2.3, o uso deste modelo seria mais adequado para uma quantidade pequena de amostras (normalmente <50).

4. Modelos Avaliados

4.1. Classificação

4.1.1 KNN (KNearest Neighbors)

O KNN foi escolhido como modelo de referência (baseline) por sua simplicidade e interpretabilidade. Foi aplicado o StandardScaler para a normalização, via Pipeline antes

Grade de hiperparâmetros

Hiperparâmetro	Valores a testar	Significado
n_neighbors	3, 5, 7, 9, 11	Quantos vizinhos consultar
weights	uniform, distance	Peso igual ou proporcional à distância
metric	euclidean, manhattan	Fórmula de cálculo da distância

Total de combinações a serem testadas: $5 \times 2 \times 2 = 20$ combinações. São poucas combinações e não se justifica utilizar Random Search. Grid Search é o método mais adequado visto que o espaço de busca é pequeno.

Melhor combinação encontrada:

- n_neighbors = 11
- weights = uniform
- metric = Euclidean

⇒ Métrica de seleção: F1-Score Macro (média dos 5 folds) F1-Macro CV: 0.9002

“11 vizinhos com peso uniforme e distância euclidiana. Pesos uniformes indicam que todos os vizinhos têm igual influência, independente da distância”

4.1.2. Árvore de Decisão (Decision Tree)

A Árvore de Decisão foi escolhida por sua alta interpretabilidade, por representar regras de negócio explícitas e legíveis através de perguntas com resultados binários. Tem o risco de overfitting, por isto, escolhidos os hiperparâmetros “clássicos” desta.

Grade de hiperparâmetros:

Hiperparâmetro	Valores a testar	Significado
max_depth	3, 5, 7, 10, None	Profundidade máxima da árvore
min_samples_leaf	1, 2, 4	Mínimo de amostras por folha
criterion	gini, entropy	Critério de pureza para os cortes

Total de combinações testadas: $5 \times 3 \times 2 = 30$ combinações $\Rightarrow$ Grid Search

Melhor combinação encontrada:

- max_depth = 7
- min_samples_leaf = 1
- criterion = entropy

$\Rightarrow$ Métrica de seleção: F1-Score Macro (média dos 5 folds) F1-Macro CV: 0.9397

“Profundidade 7 foi suficiente para capturar a estrutura dos dados sem overfitting, o que foi confirmado pelo F1Macro superior no teste (0.9589) em relação ao CV (0.9397). O critério entropy (ganho de informação) superou o gini, indicando que as divisões baseadas em redução de incerteza foram mais eficazes para separar as três classes de risco”

4.1.3 Random Forest

Foi escolhido por ser conhecido tradicionalmente com o modelo mais robusto dentre o conjunto dos três apresentados.

Grade de hiperparâmetros:

Hiperparâmetro	Valores a testar	Significado
n_estimators	100, 200, 300	Número de árvores no ensemble (votação)
max_depth	5, 10, None	Profundidade máxima de cada árvore
min_samples_leaf	1, 2, 4	Mínimo de amostras por folha

Total de combinações testadas: $3 \times 3 \times 3 = 27$ combinações $\Rightarrow$ Grid Search.

Melhor combinação encontrada:

- n_estimators = 300
- max_depth = None
- min_samples_leaf = 1

$\Rightarrow$ Métrica de seleção: F1-Score Macro (média dos 5 folds) F1-Macro CV: 0.9348

“300 árvores sem limite de profundidade indicam que o modelo se beneficiou de maior complexidade, onde cada árvore pode crescer livremente, mas o efeito de ensemble previne overfitting. O resultado inferior à Árvore de Decisão na classificação (0.9505 vs 0.9589) é

explicado pelo caráter determinístico dos dados: a estrutura matemática simples (soma dos scores) é mais facilmente capturada por uma única árvore bem calibrada do que por um ensemble que introduz aleatoriedade desnecessária

4.2. Regressão

4.2.1.KNN

Motivo da escolha pelo mesmo raciocínio do classificador: simplicidade e interpretabilidade. Na regressão é calculada a média do total_score dos K vizinhos mais próximos para previsão. Foi aplicado o StandardScaler para a normalização, via Pipeline antes.

Grade de hiperparâmetros (a mesma da classificação):

Hiperparâmetro	Valores a testar	Significado
n_neighbors	3, 5, 7, 9, 11	Quantos vizinhos consultar
weights	uniform, distance	Peso igual ou proporcional à distância
metric	euclidean, manhattan	Fórmula de cálculo da distância

Total de combinações testadas: $5 \times 2 \times 20$ combinações=> Grid Search.

Melhor combinação encontrada:

- n_neighbors = 11
- weights = distance
- metric = euclidean

⇒ RMSE negativo (neg_root_mean_squared_error) MAE teste: 24.19 pontos | R^2 : 0.9719

“Diferente da classificação, o peso distance foi preferido ao uniform, ou seja, vizinhos mais próximos recebem maior influência na média do score previsto”

4.2.2. Árvore de Decisão

Produz previsões numéricas pelo score médio das empresas na mesma folha.

Grade de hiperparâmetros:

Hiperparâmetro	Valores a testar	Significado
max_depth	3, 5, 7, 10, None	Profundidade máxima da árvore
min_samples_leaf	1, 2, 4	Mínimo de amostras por folha

⇒ *Os mesmos utilizados na classificação, mas retirando o hiperparâmetro criterion pois a regressão usa MSE por padrão.*

Total de combinações testadas: $5 \times 3 = 15$ combinações=>Grid Search.

Melhor combinação encontrada:

max_depth = 10

min_samples_leaf = 2

⇒ Métrica de seleção: RMSE negativo (neg_root_mean_squared_error) MAE teste: 18.72 pontos | R^2 : 0.9785

“Profundidade de 10 foi maior que na classificação (7). A regressão exigiu cortes mais refinados para aproximar valores numéricos contínuos. O min_samples_leaf=2 evita folhas com apenas 1 empresa, reduzindo overfitting. A melhora expressiva sobre o KNN Regressor (MAE 18,7 vs 24,2)”

4.2.3. Random Forest

O Random Forest Regressor costuma ser um ótimo candidato para regressão. Para regressão, o ensemble calcula a média das previsões de várias árvores independentes, produzindo estimativas mais suaves e precisas do que uma única árvore.

Grade de hiperparâmetros (a mesma da classificação):

Hiperparâmetro	Valores a testar	Significado
n_estimators	100, 200, 300	Número de árvores no ensemble (votação)
max_depth	5, 10, None	Profundidade máxima de cada árvore
min_samples_leaf	1, 2, 4	Mínimo de amostras por folha

Total de combinações testadas: $3 \times 3 \times 3 = 27$ combinações=>Grid Search.

Melhor combinação encontrada (a mesma da classificação):

- n_estimators = 300
- max_depth = None
- min_samples_leaf = 1

⇒ Métrica de seleção: RMSE negativo (neg_root_mean_squared_error) MAE teste: 12.07 pontos | RMSE: 27.71 | R^2 : 0.9844

“ Mesma configuração ótima encontrada, porém com impacto superior no resultado. O R^2 de 0.9844 significa que o modelo explica 98,44% da variância do total_score, sendo o erro médio de apenas 12 pontos em uma escala de 936 pontos de amplitude, o que significa 1,3% de erro relativo.

4.2.4. Comparação Modelos Regressão

5. Comparação dos modelos

5.1. Classificação

Modelo	F1-Macro	Recall Alto Risco	Acurácia	Desempenho
KNN	0.9285	0.9753	0.9355	3º
Árvore de Decisão	0.9589	0.9753	0.9585	1º
Random Forest	0.9505	0.9753	0.9539	2º

- ⇒ Foi decidido que o recall (sensibilidade) mais crítico seria para a classe de Alto Risco, uma vez que havendo falso negativo, ou seja, classificar uma empresa de Alto Risco como Risco Moderado ou Baixo Risco é um erro com consequências mais graves caso o risco venha a se materializar, do que um falso positivo, ou seja, classificar uma empresa de Baixo Risco como Alto Risco, que apesar de injusta classificação, não há riscos reais. O recall da classe Alto Risco, portanto, mede a capacidade do modelo de detectar todos os fornecedores verdadeiramente críticos.
- ⇒ Foi utilizado o F1-Macro. Ele representa a média do F1 de cada classe. É a melhor métrica global para garantir que o algoritmo seja penalizado severamente se decidir ignorar uma classe menor para focar nas maiores uma vez que as divisões não estão totalmente balanceadas.

5.2. Regressão

Modelo	MAE	RMSE	R²	Desempenho
KNN	24.19 pts	37.25 pts	0.9719	3º
Árvore de Decisão	18.72 pts	32.53 pts	0.9785	2º
Random Forest	12.07pts	27.71 pts	0.9844	1º

6. Integração com MLflow

6.1. Instalação do MLflow

O MLflow foi instalado via pip através do Terminal no ambiente virtual do projeto e configurado com backend *SQLite*(recomendado para as versões recentes).

6.2. Organização dos Experimentos

Foram criados dois experimentos separados conforme as duas tarefas de *Machine Learning* do projeto:

- **esg_classification:** Agrupa os *runs* de classificação de risco ESG (KNN, Árvore de Decisão, Random Forest)
- **esg_regression:** Agrupa os runs de previsão do score numérico total_score (KNN, Árvore e Random Forest Regressores)

Cada *run* corresponde a um modelo treinado com o *GridSearchCV* fazendo resgistros automáticos dos parâmetros e registros.

6.3. Registro de Experimentos

Na sessão 8.2 de anexos, encontram-se os registros de captura de diversas telas geradas pelo MLflow.

- Parâmetros registrados:

Parâmetro	Exemplo	Significado
best_classificador__criterion	entropy	Critério de divisão da Árvore
best_classificador__max_depth	7	Profundidade máxima
best_classificador__n_neighbors	11	Número de vizinhos do KNN
cv	StratifiedKFold(n_splits=5)	Estratégia de validação usada

- Métricas registradas:

CLASSIFICAÇÃO		
Métrica	Valor (Árvore)	Significado
test_f1_macro	0.9589	F1 médio entre as 3 classes no teste
test_f1_alto_risco	0.9693	F1 específico da classe Alto Risco
test_f1_risco_moderado	0.9473	F1 específico do Risco Moderado
test_f1_baixo_risco	0.96	F1 específico do Baixo Risco
best_cv_score	0.9397	Melhor F1-Macro na validação cruzada

REGRESSÃO		
Métrica	Valor (RF)	Significado
test_mae	12.07	Erro médio absoluto em pontos
test_rmse	27.71	Erro quadrático médio
test_r2	0.9844	Variância explicada pelo modelo

- Artefatos salvos:

Artefatos são os arquivos gerados por cada experimento. Seu valor está em podem ser carregados diretamente no dashboard sem precisar retreinar o modelo.

Artefato	Formato	Conteúdo
training_confusion_matrix.png	Imagem	Matriz de confusão do treino
estimator.html	HTML	Visualização do Pipeline completo
cv_results.csv	Tabela	Resultados de todas as combinações do GridSearch
best_estimator	Modelo	O pipeline treinado pronto para uso

7. Discussão dos resultados

Os modelos treinados se mostraram capazes de classificar fornecedores em Alto/Moderado/Baixo Risco, prever o score numérico ESG com erro médio de 12 pontos. Atendem aos objetivos e estão prontos para serem colocados em produção e alimentar a Matriz de Criticidade, Mapa de riscos ESG e o Dashboard do BI.

7.1. Modelo com melhor desempenho

- **Classificação:** Árvore de Decisão (F1-Macro = 0.9589)
- **Regressão:** Random Forest ($R^2 = 0.9844$, MAE = 12 pontos)

7.2. Overfitting e underfitting

Não houve *overfitting*, todos os modelos tiveram o F1-Macro no teste > F1-Macro no CV de treino ou seja, houve uma boa generalização. A Árvore de Decisão com max_depth=7 foi suficientemente .

Também não houve *underfitting*, uma vez que todos os modelos atingiram F1-Macro acima de 0.90. Em pesquisa na literatura, não há um consenso no “pisso” para que o F1-Macro esteja fora do *underfitting*. Contudo, pode-se considerar que em geral, valores abaixo de 0.6070 estariam em *underfitting*.

7.3. Performance

A alta performance pode ser explicada pela qualidade do dataset, o target risk_level ser derivado do total_score, e que é a soma exata das três features numéricas (environment_score + social_score + governance_score) de forma que os algoritmos puderam rapidamente identificar a relação matemática.

7.4. Métrica mais relevante

O F1-Score Macro foi a métrica principal para classificação, pois penaliza o modelo caso negligencie qualquer uma das três classes, especialmente a de Baixo Risco, que foi a classe minoritária conforme separação de pontuação escolhida. Para regressão, o MAE foi mais interpretável no contexto de negócio, pois um erro médio de 12 pontos em uma escala de 936 pontos de amplitude representa precisão de 98,7%.

7.5. Limitações encontradas

A amplitude possível para o total_score possivelmente vai de 0 a 2400. Entretanto, não haviam exemplos com pontuação acima de 1536 e também tínhamos apenas duas classes na classificação final do dataset: *medium* e *high*, não sendo possível definir com exatidão onde

ficariam os limites entre todas as classes possíveis para *low*, *medium*, *high* e *excellent*. Procurar definir onde ficariam estes limites para toda a amplitude possível, de até 2400 pontos, também nos levaria a um alto desbalanceamento entre as classes do dataset atual.

7.6. Próximos passos

Selecionado os modelos, será criado o dashboard para as visualizações das métricas, principais KPIs, Matriz de Criticidade e Mapa de Riscos ESG.

O projeto final será “containerizado” via Docker e toda a documentação será revista, melhorada e preparada para entrega final.

8. Anexos

8.1. Definição do intervalo da classe de risco (*risk_level*) a partir da pontuação total (*total_score*)

8.1.1 Premissas

Classes:

- A – Excellent
- B – High
- C – Medium
- D – Low
-

Mapeamento das fronteiras blocos de pontuação:

- Limite Superior: 2400
- Fronteira B/C: 900
- Limite Inferior: 0

Isso divide problema em dois blocos de pontuação:

- Bloco Superior (Classes A e B): De 900 a 2400. Tamanho total = 1500.
- Bloco Inferior (Classes C e D): De 0 a 900. Tamanho total = 900.

8.1.2. Cálculos dos intervalos para 4 classes

Chamando o tamanho de cada intervalo de I_A , I_B , I_C e I_D . Se eles seguem uma curva exponencial (uma progressão geométrica), logo, existe uma razão constante r entre eles, de modo que:

$$\begin{aligned}I_B &= I_A \times r \\I_C &= I_A \times r^2 \\I_D &= I_A \times r^3\end{aligned}$$

Uma vez que os tamanhos blocos são conhecidos:

$$\begin{aligned}I_B + I_A &= 1500 \\I_D + I_C &= 900\end{aligned}$$

Criamos um sistema de equações, que reescrevendo para duas variáveis I_A e r :

$$\begin{aligned}(\text{I}) \quad I_A(1 + r) &= 1500 \\(\text{II}) \quad I_A \times r^2(1 + r) &= 900\end{aligned}$$

Simplificando a equação, eliminando $I_A(1 + r)$, obtemos $r^2 = 0.6 \approx 0.775$.

Uma vez obtida a razão, basta substituir na equação (I), $I_A \approx 845$.

Tendo o valor de I_A , basta multiplicarmos seguidamente pela razão para obtermos I_B , I_C e I_D .

$$\begin{aligned}I_B &\approx 655 \\I_C &\approx 507 \\I_D &\approx 393\end{aligned}$$

Logo, a divisão entre as classes teria o seguinte comportamento:

level	total_score
excellent	1555-2400
high	900-1554
medium	393-899
low	0-392

8.1.3 Cálculo dos intervalos para classes

Uma vez que em nosso projeto é exigido que haja apenas 3 classes para risco, iremos agora fazer o cálculo inverso, usando a mesma razão de 0.775, porém agora com apenas 3 intervalos:

$$\begin{aligned}I_C + I_B + I_A &= 1500 \\I_B &= I_A \times r \\I_C &= I_A \times r^2\end{aligned}$$

Substituindo,

$$\begin{aligned}I_A \cdot (1 + r + r^2) &= 1500 \\I_A &\approx 1010\end{aligned}$$

Tendo o valor de I_A , basta multiplicarmos seguidamente pela razão para obtermos I_B e I_C :

$$\begin{aligned}I_B &\approx 783 \\I_C &\approx 606\end{aligned}$$

Assim, os nossos limites finais terá a seguinte divisão, de forma a ter um comportamento semelhante aos identificados no *dataset*, ou seja, gap exponencial a uma razão fixa de 0.775:

risk_level	total_score
low	1389-2400
medium	606-1388
high	0-605

8.2. Capturas de tela no MLflow

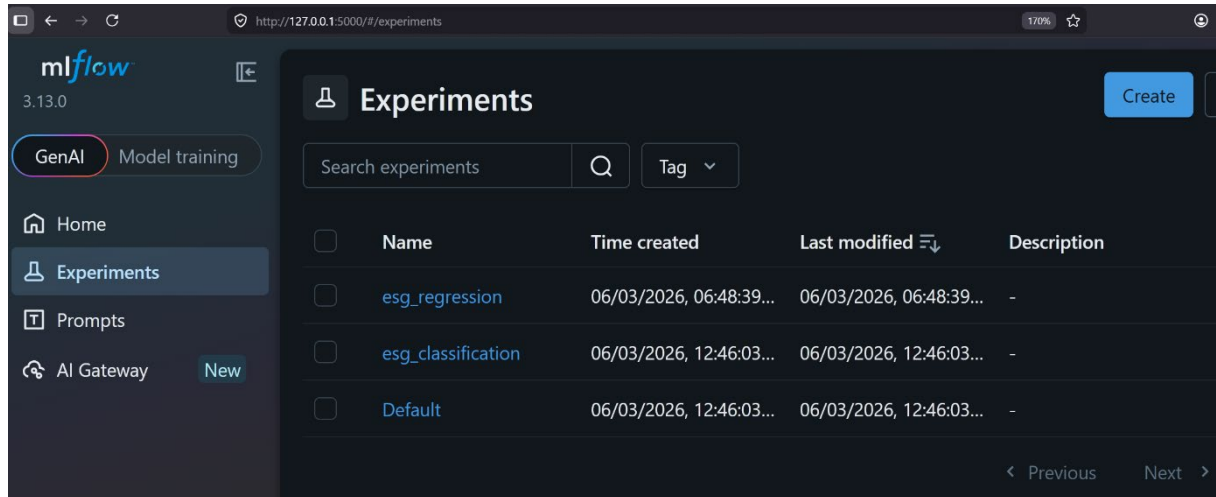


Figura 1- Dashboard com experimentos realizados

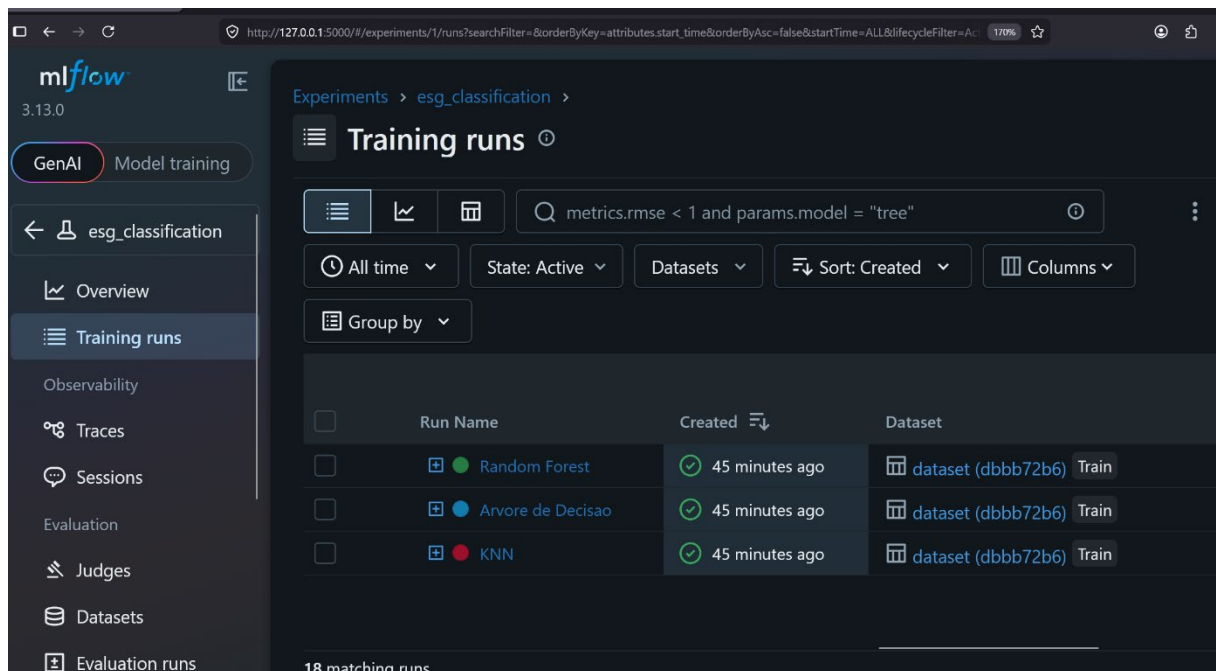


Figura 2- Runs deClassificação

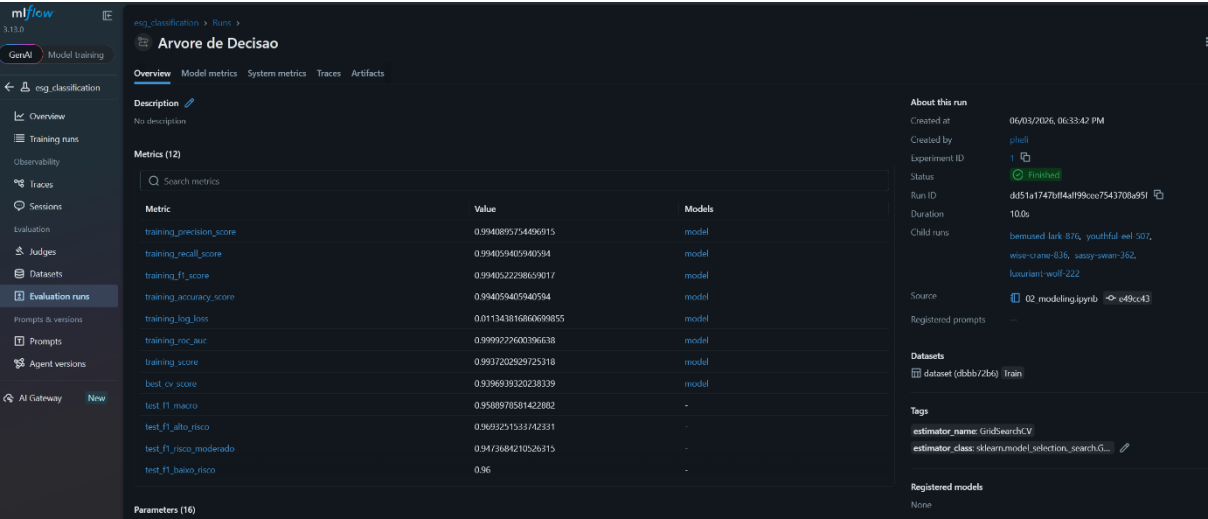


Figura 3 - Detalhes Árvore de Decisão (melhor resultado na Classificação)

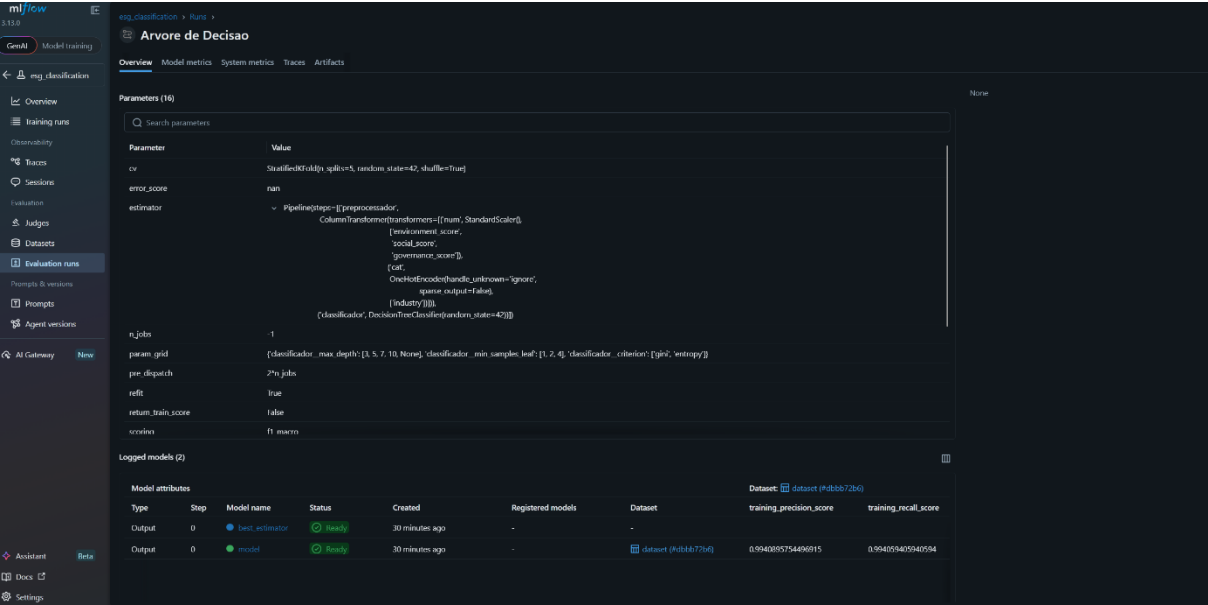


Figura 4 - Detalhes Árvore de Decisão (melhor resultado na Classificação)

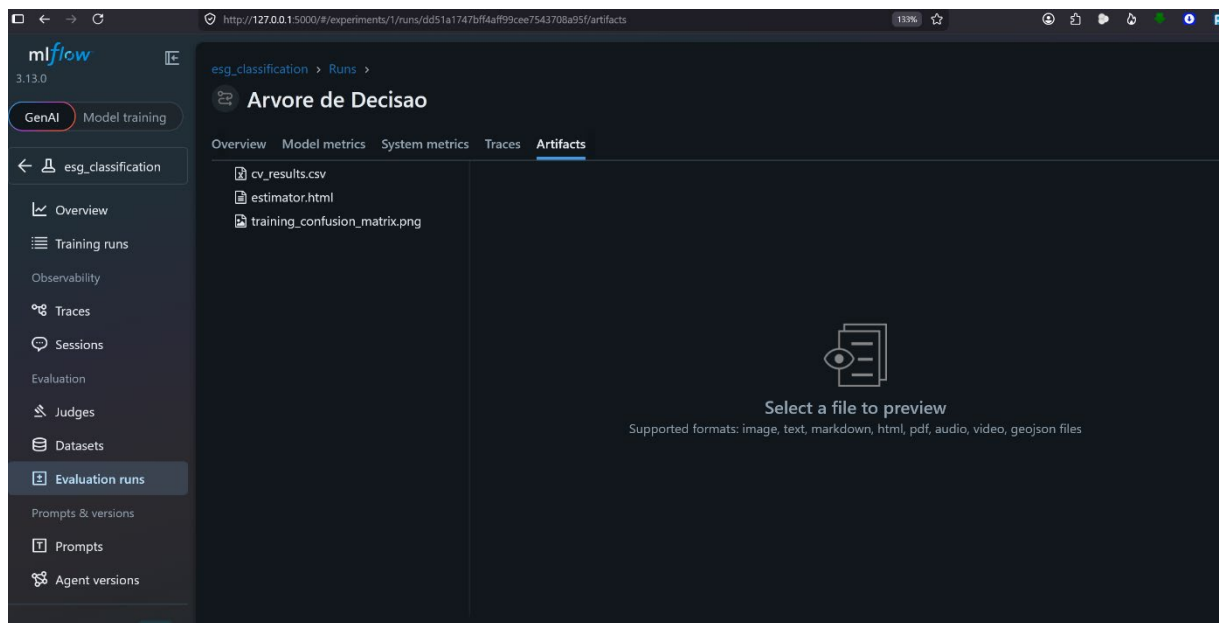


Figura 5 - Artefatos Árvore de Decisão (melhor resultado na Classificação)

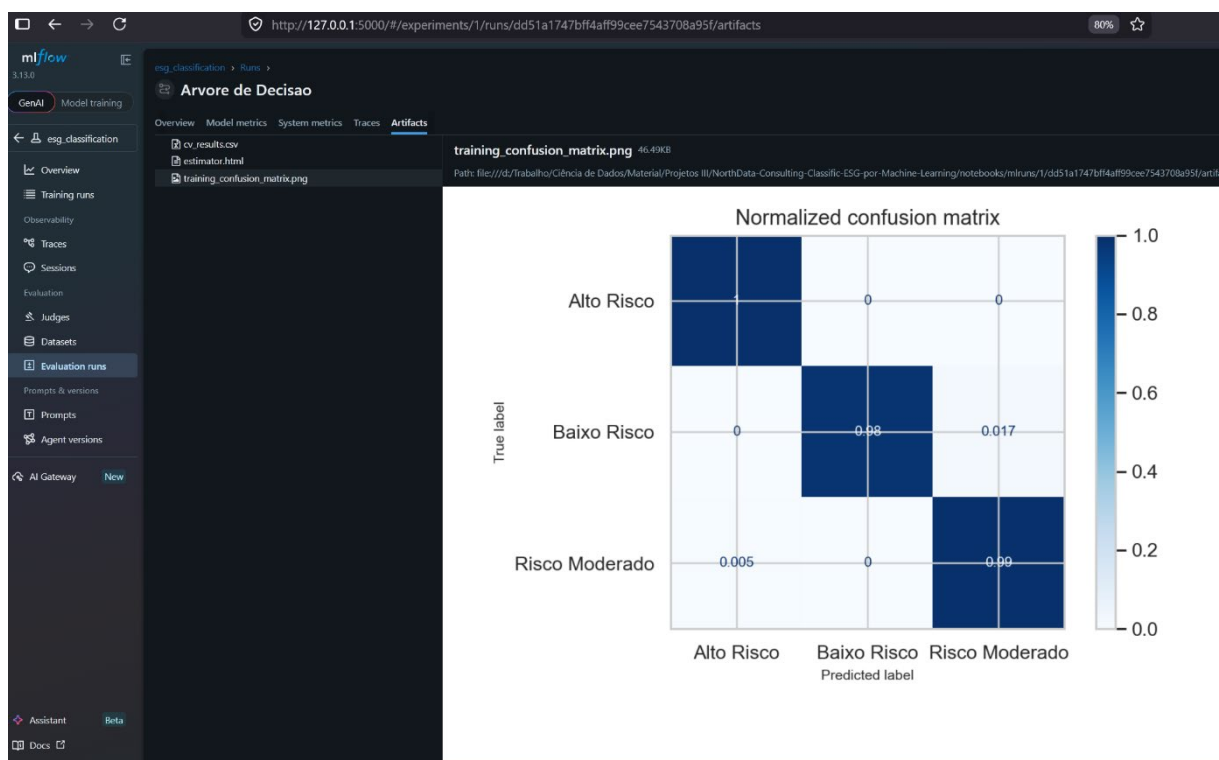


Figura 6 - Matriz de Confusão Árvore de Decisão (melhor resultado na Classificação)

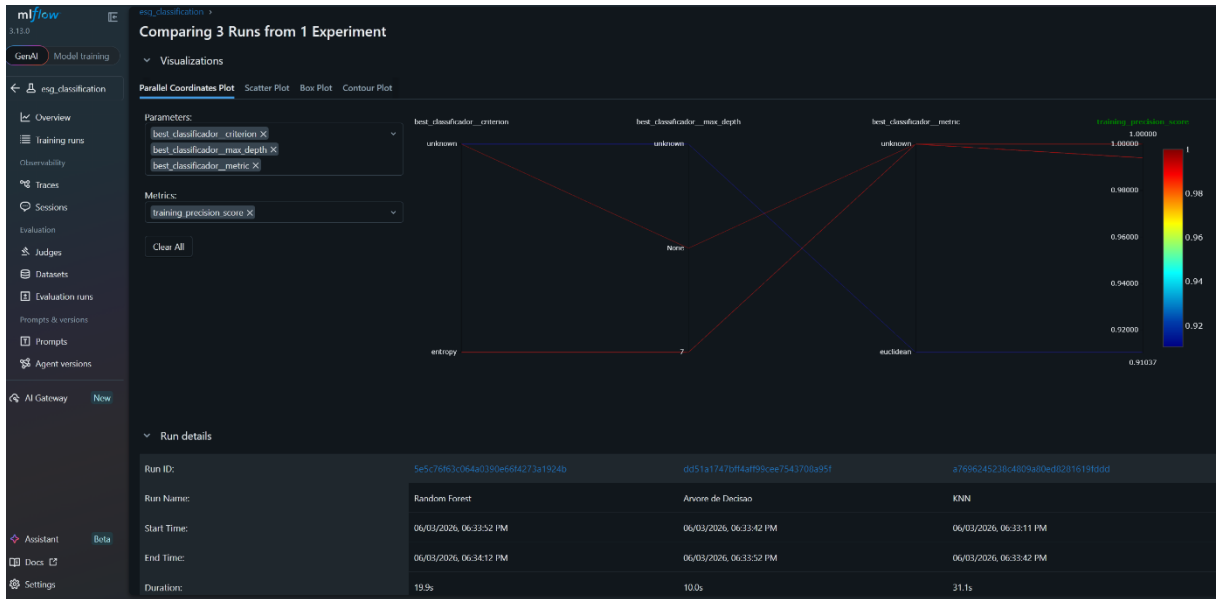


Figura 7- Comparação modelos classificação

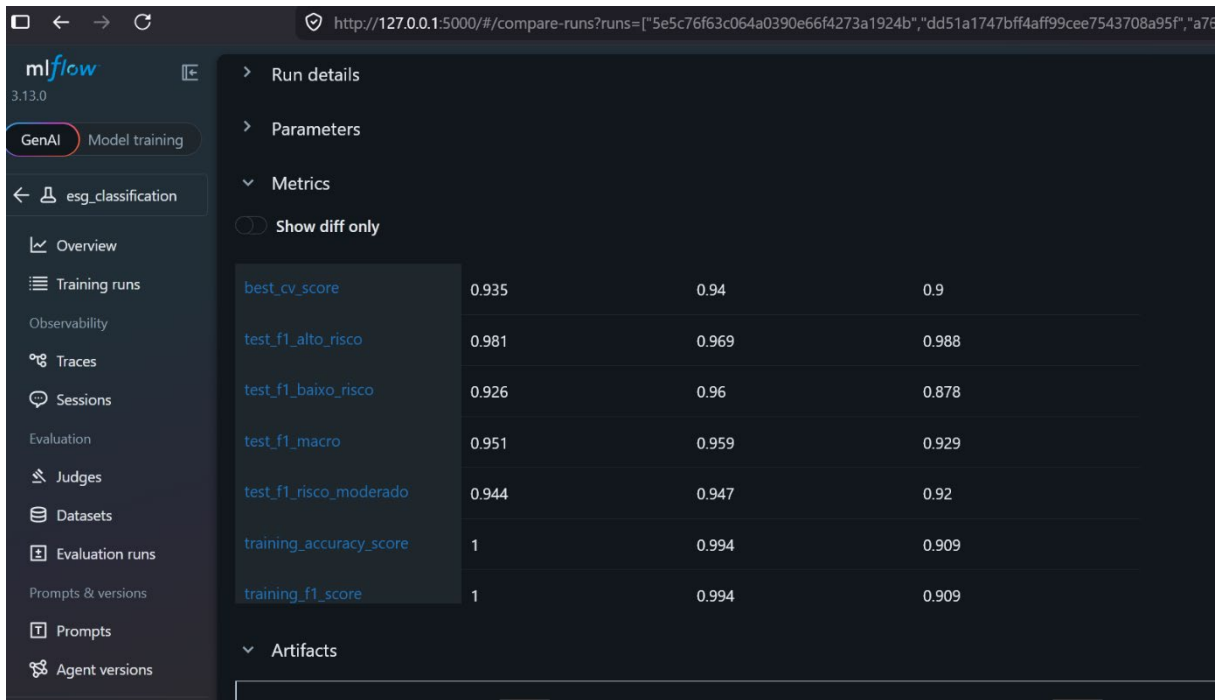


Figura 8 - Comparação modelos classificação com foco no F1

The screenshot shows the 'Training runs' page in the mlflow interface. The left sidebar contains navigation options like Overview, Training runs, and various evaluation tools. The main area displays a table of runs with columns for Run Name, Created time, Dataset, Duration, Source, and Models. A search filter is applied: 'metrics.rmse < 1 and params.model = "tree"'. The table lists several runs, including 'clean-crane-639', 'brawny-mole-905', and 'Random Forest Regr...'. The 'Random Forest Regr...' run is highlighted as the best estimator.

Run Name	Created	Dataset	Duration	Source	Models
clean-crane-639	33 minutes ago	dataset (dbbb72b6) Train	6.2s	02_mod...	model
brawny-mole-905	34 minutes ago	dataset (dbbb72b6) Train	6.4s	02_mod...	model
Random Forest Regr...	38 minutes ago	dataset (dbbb72b6) Train	15.5s	02_mod...	best_estimator
Arvore de Decisao Re...	38 minutes ago	dataset (dbbb72b6) Train	9.0s	02_mod...	best_estimator
KNN Regressor	38 minutes ago	dataset (dbbb72b6) Train	23.0s	02_mod...	best_estimator

Figura 9- Runs de Regressão

The screenshot shows the 'Evaluation runs' page for the 'Random Forest Regressor'. The left sidebar is the same as in Figure 9. The main area displays a table of metrics and parameters for the selected run. The 'Metrics' table lists various performance metrics like training_mean_squared_error, training_mean_absolute_error, training_f2_score, training_root_mean_squared_error, training_score, best_cv_score, test_mae, test_rmse, and test_f2. The 'Parameters' table shows the 'cv' parameter set to 5. The right sidebar provides additional information about the run, including its creation time, creator, experiment ID, status (Finished), run ID, duration, child runs, source, registered prompts, datasets, tags, and registered models.

Metric	Value	Models
training_mean_squared_error	59.67502189218912	model
training_mean_absolute_error	4.099254125412535	model
training_f2_score	0.99873305806266	model
training_root_mean_squared_error	7.724960963797106	model
training_score	-7.724960963797106	model
best_cv_score	-21.409920549347945	model
test_mae	12.073410138248848	-
test_rmse	27.705829734553742	-
test_f2	0.9844292976456975	-

Parameter	Value
cv	5

Figura 10- Detalhes Random Forest(melhor resultado naRegressão)

mlflow 3.13.0

GenAI Model training

esg_regression

Overview Training runs Observability Traces Sessions Evaluation Judges Datasets Evaluation runs Prompts & versions Prompts Agent versions AI Gateway New Assistant Beta Docs Settings

esg_regression > Runs >

Random Forest Regressor

Overview Model metrics System metrics Traces Artifacts

Search parameters

Parameter	Value
cv	5
error_score	nan
estimator	Pipeline(steps=[('preprocessor', ColumnTransformer(transformers=[('num', StandardScaler()), ('environment_score', 'social_score', 'governance_score']], ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), ['industry'])))], ('regressor', RandomForestRegressor(random_state=42)))]
n_jobs	-1
param_grid	{'regressor__n_estimators': [100, 200, 300], 'regressor__max_depth': [5, 10, None], 'regressor__min_samples_leaf': [1, 2, 4]}
pre_dispatch	2*n_jobs
refit	True
return_train_score	False
scoring	neg_root_mean_squared_error
verbose	0
best_regressor__max_depth	None
best_regressor__min_samples_leaf	1
best_regressor__n_estimators	300

Registered models

None

Logged models (2)

Type	Step	Model name	Status	Created	Registered models	Dataset
------	------	------------	--------	---------	-------------------	---------

Figura 11- Detalhes Random Forest (melhor resultado na Regressão)

mlflow 3.13.0

GenAI Model training

esg_regression

Overview Training runs Observability Traces Sessions Evaluation Judges Datasets Evaluation runs Prompts & versions Prompts Agent versions AI Gateway New Assistant Beta Docs Settings

esg_regression > Runs >

Random Forest Regressor

Overview Model metrics System metrics Traces Artifacts

cv_results.csv estimator.html

estimator.html 139.84KB

Path: file:///d:/Trabalho/Ciência de Dados/Material/Projetos III/NorthData-Consulting-Classific-ESG-por-M

```
graph TD
    GridSearchCV --> best_estimator_Pipeline
    subgraph best_estimator_Pipeline [best_estimator_: Pipeline]
        direction TB
        preprocessor_ColumnTransformer[preprocessor: ColumnTransformer] --> RandomForestRegressor
        preprocessor_ColumnTransformer --> num_StandardScaler[num: StandardScaler]
        preprocessor_ColumnTransformer --> cat_OneHotEncoder[cat: OneHotEncoder]
        num_StandardScaler --> 3_features[3 features]
        cat_OneHotEncoder --> 44_features[44 features]
        3_features --> 47_features[47 features]
        44_features --> 47_features
    end
    47_features --> RandomForestRegressor
```

Figura 12- Artefatos Random Forest (melhor resultado na Regressão)

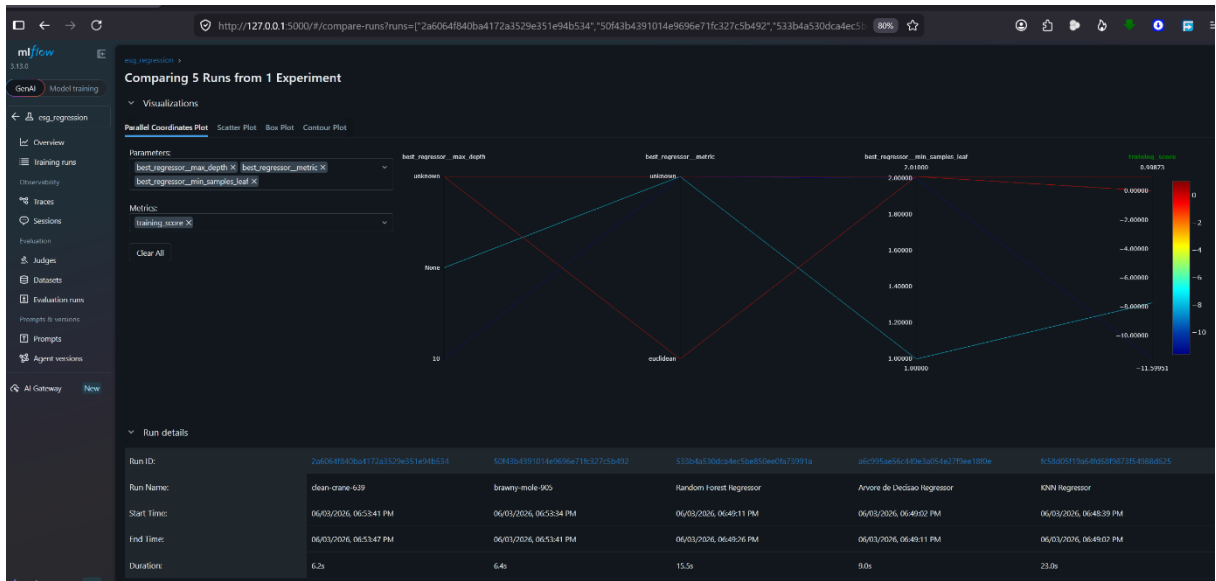


Figura 13- Comparação modelos regressão